

从调用，到工程。

智能体时代的技能化开发模式

起点：被反复验证的技能

好的技能不是凭空设计出来的，而是**从实际调用中被验证出来的**。智能体每天都在被使用，而使用本身就是一场大规模的筛选。

一 看频率

哪些对话流程、提示词组合被**反复使用**——高频即需求。

二 看稳定性

哪些调用**每次都出好结果**——稳定即可依赖。

三 看完整度

哪些流程已经**形成了固定套路**——成型即可沉淀。

> 这一步的判断很简单：**先有实践，再有工程。**值得抽取的技能，一定已经在调用模式里被证明过自己。

抽取：模块化与结构化

把一段「用得好的调用」从对话流里抽出来，拆成**四个可管理的部分**——这就是技能从「经验」变成「资产」的过程。

一 输入约定

这个技能需要什么材料、什么格式，**边界清晰**，机器可校验。

二 执行步骤

先做什么、后做什么、分支怎么走，**结构化成明确流程**。

三 工具清单

要调用哪些能力，一一声明，**可授权、可审计**。

四 验收标准

什么样算做完、什么样算做对，**可判定、可复检**。

> 模块化的意义是**可单独替换**；结构化的意义是**机器可读、可校验、可自动执行**。

后台：引擎、容器、技能三层

技能层	**能力本体，可插拔的最小单元。**每个技能独立成型，按需挂载，随取随用。
容器层	**隔离与资源。**每个技能一个独立运行环境，互不干扰，出问题可单独回收。
引擎层	**调度与运行时。**负责把任务跑起来：排队、分发、监控、重试，全程自动。

> 三层各司其职：**技能管能力，容器管隔离，引擎管调度**——后台因此既灵活又稳定。

配置：灵活自动，人定标准

「灵活自动的后台配置」不是无人值守的口号，而是把**重复的配置工作交给系统**，把**标准的定义权留给人**。

一 按需挂载

任务来了，需要的技能自动就位，**不用的自动卸下**。

二 自动填充

参数依任务上下文**自动生成**，不再逐项手填。

三 自动启停

容器按负载自动启停，**资源用完即还**。

四 失败重试

执行失败自动换路重试，**人只在最后看结果**。

> 人只负责定义一件事：**什么算成功。**

前端：可阅读、可修改、可确认

后台再自动，也不能变成黑盒。前端界面必须让人**看得懂、改得动、说了算**——这是这套模式能被信任的前提。

一 可阅读

每个技能是什么、怎么运行、用了哪些工具，**一目了然**。

二 可修改

参数、流程与规则**开放编辑**，改完立即生效于下一次执行。

三 可确认

关键动作执行前停下，**人来点确认**——签字权始终在人手里。

> **信任不来自承诺，来自可见。**三个「可」，把黑盒变成白盒。

整合：一条完整的闭环链路

前端定义
人阅读并修改技能，确认规则

→

智能体编排
理解意图，选择并串联技能

→

后台执行
引擎调度，容器隔离运行

→

结果回传
产出回到前端，供人验证

→

沉淀复用
好的执行固化为新的技能

前端负责**看与决定**，后台负责**跑与交付**，智能体在中间负责**理解与编排**——三者各就其位。

> 闭环每转一圈，**技能库就厚一层**——这是这套模式与一次性脚本的本质区别。

为什么好用：四个理由

一 复用

一次沉淀，处处调用。技能不随对话消失，**成为可积累的资产**。

二 透明

执行过程全程可见，**黑盒变成白盒**，问题可定位。

三 可控

关键节点人签字，**自动化不等于失控**。

四 可迭代

每次执行都是一次检验，**技能随使用持续进化**。

四条合起来，指向同一个结论：**这是把智能体从玩具变成生产资料的那一步。**

边界：它适合做什么

适合沉淀成技能

高频、重复、有明确验收标准的工作流：报告生成、数据巡检、内容装配、批量整理——套路越固定，沉淀越划算。

不适合强行固化

高度创意、一次性、无法定义完成标准的事：留给自由对话。硬把它们做成技能，只会得到僵硬的模板。

> 判断标准一句话：**能把「什么算做对」写清楚的工作，才值得做成技能。**

把经验固化成技能，
把技能交给后台，
把决定权留给人。

智能体由此从「帮你想」，走向**「替你建」**——

这就是基于智能体的功能开发模式。